Listing 11.5 Parsing a certificate

```
public static X509Certificate decodeCert(String encodedCert)
{
    var pem = URLDecoder.decode(encodedCert, UTF_8);    #A
    try (var in = new
ByteArrayInputStream(pem.getBytes(UTF_8))) {
        var certFactory =
CertificateFactory.getInstance("X.509");    #B
        return (X509Certificate)
certFactory.generateCertificate(in);    #B
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

#A Decode the URL-encoding added by Nginx
#B Parse the PEM-encoded certificate using a CertificateFactory

There can be multiple SAN entries in a certificate and each entry can have a different type. Mkcert uses the DNS type, so the code looks for the first DNS SAN entry and returns that as the name. Java returns the SAN entries as a collection of 2-element `List` objects, the first of which is the type (as an integer) and the second is the actual value (either a `String` or a byte array, depending on the type). DNS entries have type value 2. If the certificate contains a matching entry then you can set the client ID as the subject attribute on the request, just as you've done when authenticating users. Because the trusted CA issues client certificates, you can instruct the CA not to issue a certificate that clashes with the name of an existing user. Open the